

9 AI Concepts That Put You Ahead of 99%

A field guide to how production AI systems are actually built in 2026: the runtime, the controls, and the discipline that separates hobby projects from real systems.

🔄 Agent Runtime

01-03

🔧 Control Plane

04-06

🛡️ Reliability Layer

07-09

Swipe through all 9 →

01

Agentic Loops

Build intelligent agents that actually get work done

Most “AI agents” are really just one loop, run with discipline.

Plan what to do. Act on it. Observe what happened. Reflect on whether the goal is actually met. Repeat until it is. That's the entire trick: a junior analyst who checks their own work before handing it back, instead of guessing once and walking away.

TAKEAWAY

A vague exit condition is the #1 cause of agents that loop forever, or quit too early.

Swipe →

02

MCP (Model Context Protocol)

Build intelligent agents that actually get work done

Every model used to need its own custom bridge to every tool. That doesn't scale.

MCP is the shared, open standard that fixes it: one interface any model can use to call any tool, database, or file system. Think of it as the USB-C port for AI: plug in once, work with everything.

TAKEAWAY

Give an agent 50 tools it barely understands and it performs worse, not better. Relevance beats volume.

Swipe →

03

Subagents & Multi-Agent Systems

Build intelligent agents that actually get work done

One agent juggling ten jobs is just a worse version of ten focused agents.

Multi-agent systems split the problem across specialists (a researcher, a coder, a reviewer), each running its own context and reasoning loop, then hand off and merge results like an actual team would.

TAKEAWAY

More agents means more coordination overhead. The bottleneck moves from thinking to managing.

Swipe →

04

AI Gateway

Operate, protect, and optimize AI at scale

Ask most teams which models are being called across their company right now. Most can't answer.

An AI gateway is the single choke point every request passes through: routing traffic, enforcing spend and safety policy, rotating keys, logging usage. Portkey, LiteLLM, and Kong AI Gateway all exist to be that layer.

TAKEAWAY

Without a gateway, cost and risk stay invisible until the invoice or the incident arrives.

Swipe →

05

Inference Economics

Operate, protect, and optimize AI at scale

A clever prompt still costs money by the token, not by how clever it sounds.

Three levers actually move the bill: prompt caching reuses context you've already paid for, batching groups requests for cheaper throughput, and routing sends easy tasks to smaller, cheaper models automatically instead of your best (and priciest) model.

TAKEAWAY

A 50,000-token task run 10,000 times a day is a line item on the budget, not a rounding error.

Swipe →

06

Guardrails

Operate, protect, and optimize AI at scale

The seatbelt your users never notice, until the one day they need it.

Guardrails check every input before it reaches the model and every output before it reaches the user: blocking prompt injection, filtering PII, catching jailbreak attempts, moderating unsafe results at every boundary, not just at the edges.

TAKEAWAY

Guardrails are cheap to design in on day one, and expensive to bolt on after an incident.

Swipe →

07

Evals

Measure, observe, and keep improving

You can't write a unit test that asserts a model's output is 'correct.'

So evals score quality, accuracy, and tone at scale instead: structured tests that run automatically before every change ships. Braintrust, Galileo, and Arize Phoenix exist purely to make this measurable rather than a vibe check.

TAKEAWAY

If you can't measure a regression, your users will find it for you, in production.

Swipe →

08

Observability

Measure, observe, and keep improving

A multi-agent system fails in ways a normal stack trace can't explain.

Observability captures the full trace of a request: every tool call, token, latency spike, handoff between agents. So when something breaks at 2am, you can see exactly where and why, instead of guessing across five disconnected logs.

TAKEAWAY

The best teams debug from one unified trace. Everyone else is guessing.

Swipe →

09

The Bitter Lesson

Measure, observe, and keep improving

Richard Sutton's idea, and quietly the most important one on this list.

Decades of AI research show the same pattern on repeat: hand-crafted rules and human cleverness lose to general methods that scale with more data and more compute. Not because clever is wrong. It's because it doesn't compound the way scale does.

TAKEAWAY

Before you engineer a clever workaround, ask if more data or compute solves it more durably.

Swipe for the close →

Reliable. Observable. Cost-aware.

Most teams get the first layer right and stop. The ones actually shipping AI in production have all three. A demo only needs to work once, and a real system has to work every time.

🔄 Agent Runtime ✓ complete

⚙️ Control Plane ✓ complete

🛡️ Reliability Layer ✓ complete

Which of these nine is your team missing?

Save this for your next architecture review.

Repost if your team needs to see it.

Comment with the one you'd add.

Follow for more breakdowns like this.